```
#include <stdio.h>
#include <string.h>

char stack[50], input[50];
int top = -1, ip = 0;

char table[8][8] = {
   {'>','>','<','<','<','>','<','>'},
   {'>','>','<','<','<','>','<','>'},
   {'>','>','>','>','<','>','<','>'},
   {'>','>','>','>','<','>','<','>'},
   {'<','<','<','<','<','=','<',' '},
   {'>','>','>','>',' ','>',' ','>'},
   {'>','>','>','>',' ','>',' ','>'},
   {'<','<','<','<','<',' ','<',' '}
};

int findex(char c)
{
   switch(c)
   {
      case '+': return 0;
      case '-': return 1;
      case '*': return 2;
      case '/': return 3;
      case '(': return 4;
      case ')': return 5;
      case 'i': return 6;
      case '$': return 7;
   }
   return -1;
}

void display()
{
   int i;
   for(i = 0; i <= top; i++)
      printf("%c", stack[i]);

   printf("\t\t%s\n", input + ip);
}

int reduce()
{
```

```
    if(top >= 0 && stack[top] == 'i')
    {
        stack[top] = 'E';
        return 1;
    }

    if(top >= 2 &&
       stack[top] == ')' &&
       stack[top-1] == 'E' &&
       stack[top-2] == '(')
    {
        top -= 2;
        stack[top] = 'E';
        return 1;
    }

    if(top >= 2 &&
       stack[top] == 'E' &&
       stack[top-2] == 'E')
    {
        if(stack[top-1] == '+' ||
           stack[top-1] == '-' ||
           stack[top-1] == '*' ||
           stack[top-1] == '/')
        {
            top -= 2;
            stack[top] = 'E';
            return 1;
        }
    }

    return 0;
}

int main()
{
    char a, b;
    int i;

    printf("Enter expression (use i for id): ");
    scanf("%s", input);
    strcat(input, "$");

    stack[++top] = '$';
```

```
printf("\nSTACK\t\tINPUT\n");

while(1)
{
   i = top;

   while(i >= 0 && stack[i] == 'E')
      i--;

   a = stack[i];
   b = input[ip];

   if(top == 1 && stack[0] == '$' &&
     stack[1] == 'E' && b == '$')
   {
      printf("\nAccepted\n");
      break;
   }

   if(findex(a) == -1 || findex(b) == -1)
   {
      printf("\nRejected\n");
      break;
   }

   if(table[findex(a)][findex(b)] == '<' ||
     table[findex(a)][findex(b)] == '=')
   {
      stack[++top] = b;
      ip++;
      display();
   }
   else if(table[findex(a)][findex(b)] == '>')
   {
      if(!reduce())
      {
         printf("\nRejected\n");
         break;
      }
      display();
   }
   else
   {
```

```
            printf("\nRejected\n");
            break;
        }
    }

    return 0;
}
```

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

/*
Grammar:
E  -> T E'
E' -> + T E' | - T E' | #
T  -> F T'
T' -> * F T' | / F T' | #
F  -> ( E ) | id
*/

char input[50];
int pos = 0;

char lookahead()
{
    return input[pos];
}

void match(char c)
{
    if(input[pos] == c)
        pos++;
    else
    {
        printf("Error at position %d: expected '%c', got '%c'\n",
               pos, c, input[pos]);
        exit(1);
    }
}
```

```
void E();
void EP();
void T();
void TP();
void F();

void E()
{
   printf("E -> T E'\n");
   T();
   EP();
}

void EP()
{
   if(lookahead() == '+')
   {
      printf("E' -> + T E'\n");
      match('+');
      T();
      EP();
   }
   else if(lookahead() == '-')
   {
      printf("E' -> - T E'\n");
      match('-');
      T();
      EP();
   }
   else
   {
      printf("E' -> #\n");
   }
}

void T()
{
   printf("T -> F T'\n");
   F();
   TP();
}

void TP()
```

```
{
   if(lookahead() == '*')
   {
      printf("T' -> * F T'\n");
      match('*');
      F();
      TP();
   }
   else if(lookahead() == '/')
   {
      printf("T' -> / F T'\n");
      match('/');
      F();
      TP();
   }
   else
   {
      printf("T' -> #\n");
   }
}

void F()
{
   if(lookahead() == '(')
   {
      printf("F -> ( E )\n");
      match('(');
      E();
      match(')');
   }
   else if(isalnum(lookahead()))
   {
      printf("F -> %c\n", lookahead());
      pos++;
   }
   else
   {
      printf("Error at position %d: unexpected '%c'\n",
            pos, lookahead());
      exit(1);
   }
}

int main()
```

```
{
    printf("Enter expression (end with $): ");
    scanf("%s", input);

    E();

    if(input[pos] == '$')
        printf("\nParsing successful!\n");
    else
        printf("\nError: unexpected character '%c'\n", input[pos]);

    return 0;
}



#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

char *p;
int tcount = 0, n = 0;

struct {
    char op[8], a1[20], a2[20], res[20];
} q[50];

void emit(char *op, char *a1, char *a2, char *res)
{
    strcpy(q[n].op, op);
    strcpy(q[n].a1, a1);
    strcpy(q[n].a2, a2);
    strcpy(q[n].res, res);
    n++;
}

void newtemp(char *t)
{
    sprintf(t, "t%d", ++tcount);
}

void skip()
{
```

```
    while(*p == ' ')
        p++;
}

void expr(char *r);

void factor(char *r)
{
    int i = 0;
    skip();

    if(*p == '(')
    {
        p++;
        expr(r);
        skip();
        if(*p == ')')
            p++;
    }
    else if(*p == '-')
    {
        char a[20];
        p++;
        factor(a);
        newtemp(r);
        emit("uminus", a, "", r);
    }
    else
    {
        while(isalnum(*p) || *p == '_')
            r[i++] = *p++;
        r[i] = '\0';
    }
}

void term(char *r)
{
    char op[2] = {0}, b[20], t[20];

    factor(r);

    for(skip(); *p == '*' || *p == '/'; skip())
    {
        op[0] = *p++;
```

```
        factor(b);
        newtemp(t);
        emit(op, r, b, t);
        strcpy(r, t);
    }
}

void expr(char *r)
{
    char op[2] = {0}, b[20], t[20];

    term(r);

    for(skip(); *p == '+' || *p == '-'; skip())
    {
        op[0] = *p++;
        term(b);
        newtemp(t);
        emit(op, r, b, t);
        strcpy(r, t);
    }
}

int main()
{
    char line[100], lhs[20] = "", res[20], *eq;

    printf("Enter an expression: ");
    fgets(line, sizeof(line), stdin);
    line[strcspn(line, "\n")] = '\0';

    p = line;
    eq = strchr(line, '=');

    if(eq)
    {
        *eq = '\0';
        sscanf(line, "%19s", lhs);
        p = eq + 1;
    }

    expr(res);
    skip();
```

```
    if(*p != '\0')
    {
        printf("Invalid expression.\n");
        return 1;
    }

    if(eq)
        emit("=", res, "", lhs);

    printf("\nThree Address Code:\n");

    for(int i = 0; i < n; i++)
    {
        if(!strcmp(q[i].op, "="))
            printf("%s = %s\n", q[i].res, q[i].a1);
        else if(!strcmp(q[i].op, "uminus"))
            printf("%s = -%s\n", q[i].res, q[i].a1);
        else
            printf("%s = %s %s %s\n",
                  q[i].res, q[i].a1, q[i].op, q[i].a2);
    }

    printf("\nQuadruples:\n");
    printf("%-6s%-8s%-8s%-8s%-8s\n",
          "No.", "Op", "Arg1", "Arg2", "Result");

    for(int i = 0; i < n; i++)
        printf("%-6d%-8s%-8s%-8s%-8s\n",
              i, q[i].op, q[i].a1, q[i].a2, q[i].res);

    return 0;
}
```

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int n, nv = 0, nc = 0;

char tac[20][50], var[30][10], code[100][40];
```

```
void addvar(char *s)
{
   int i;

   if(!isalpha(s[0]))
      return;

   for(i = 0; i < nv; i++)
      if(strcmp(var[i], s) == 0)
         return;

   strcpy(var[nv++], s);
}

void emit(char *s)
{
   strcpy(code[nc++], s);
}

int main()
{
   int i;
   char r[10], a1[10], op[5], a2[10], lbl[10],
        buf[40], *jmp;

   printf("Enter the no of three address statements:\n");
   scanf("%d", &n);

   printf("Enter the three address code:\n");

   for(i = 0; i < n; i++)
      scanf(" %[^\n]", tac[i]);

   for(i = 0; i < n; i++)
   {
      if(sscanf(tac[i], "if %s %s %s goto %s",
               a1, op, a2, lbl) == 4)
      {
         addvar(a1);
         addvar(a2);

         if(strcmp(op, "<") == 0)
            jmp = "JL";
         else if(strcmp(op, ">") == 0)
```

```
        jmp = "JG";
    else if(strcmp(op, "<=") == 0)
        jmp = "JLE";
    else if(strcmp(op, ">=") == 0)
        jmp = "JGE";
    else if(strcmp(op, "==") == 0)
        jmp = "JE";
    else
        jmp = "JNE";

    sprintf(buf, "\tMOV AX, %s", a1);
    emit(buf);

    sprintf(buf, "\tCMP AX, %s", a2);
    emit(buf);

    sprintf(buf, "\t%s %s", jmp, lbl);
    emit(buf);
}
else if(sscanf(tac[i], "goto %s", lbl) == 1)
{
    sprintf(buf, "\tJMP %s", lbl);
    emit(buf);
}
else if(tac[i][strlen(tac[i]) - 1] == ':')
{
    emit(tac[i]);
}
else if(sscanf(tac[i], "%s = %s %s %s",
            r, a1, op, a2) == 4)
{
    addvar(r);
    addvar(a1);
    addvar(a2);

    sprintf(buf, "\tMOV AX, %s", a1);
    emit(buf);

    if(op[0] == '+')
    {
        sprintf(buf, "\tADD AX, %s", a2);
        emit(buf);
    }
    else if(op[0] == '-')
```

```
        {
            sprintf(buf, "\tSUB AX, %s", a2);
            emit(buf);
        }
        else if(op[0] == '*')
        {
            sprintf(buf, "\tMOV BX, %s", a2);
            emit(buf);
            emit("\tMUL BX");
        }
        else if(op[0] == '/')
        {
            emit("\tMOV DX, 0");
            sprintf(buf, "\tMOV BX, %s", a2);
            emit(buf);
            emit("\tDIV BX");
        }

        sprintf(buf, "\tMOV %s, AX", r);
        emit(buf);
    }
    else if(sscanf(tac[i], "%s = %s", r, a1) == 2)
    {
        addvar(r);
        addvar(a1);

        sprintf(buf, "\tMOV AX, %s", a1);
        emit(buf);

        sprintf(buf, "\tMOV %s, AX", r);
        emit(buf);
    }
}

printf("\n8086 Assembly Code:\n");

printf("DATA SEGMENT\n");
for(i = 0; i < nv; i++)
    printf("\t%s DW ?\n", var[i]);

printf("DATA ENDS\n");
printf("CODE SEGMENT\n");
printf("\tASSUME CS:CODE, DS:DATA\n");
printf("START:\tMOV AX, DATA\n");
```

```
    printf("\tMOV DS, AX\n");

    for(i = 0; i < nc; i++)
        printf("%s\n", code[i]);

    printf("\tMOV AH, 4CH\n");
    printf("\tINT 21H\n");
    printf("CODE ENDS\n");
    printf("END START\n");

    return 0;
}



#include <stdio.h>
#include <string.h>
#include <ctype.h>

char p[20][20], F[26][30], G[26][30];
int n, change;

/* Add a symbol if it is not already present */
int add(char *s, char c)
{
    if(strchr(s, c))
        return 0;

    int l = strlen(s);
    s[l] = c;
    s[l+1] = '\0';

    return 1;
}

/* Find FIRST of all non-terminals */
void first()
{
    int i, j, k, nullable;

    do {
        change = 0;

        for(i = 0; i < n; i++) {
```

```
            char *s = F[p[i][0]-'A'];

            for(j = 3; p[i][j]; j++) {
                char c = p[i][j];

                if(c == '#') {
                    change |= add(s, '#');
                    break;
                }

                if(!isupper(c)) {
                    change |= add(s, c);
                    break;
                }

                for(k = 0; F[c-'A'][k]; k++)
                    if(F[c-'A'][k] != '#')
                        change |= add(s, F[c-'A'][k]);

                nullable = strchr(F[c-'A'], '#') != NULL;

                if(!nullable)
                    break;

                if(!p[i][j+1])
                    change |= add(s, '#');
            }
        }
    } while(change);
}

/* Find FOLLOW of all non-terminals */
void follow()
{
    int i, j, k, nullable;

    add(G[p[0][0]-'A'], '$');

    do {
        change = 0;

        for(i = 0; i < n; i++) {
            for(j = 3; p[i][j]; j++) {
                char c = p[i][j], *s;
```

```
            if(!isupper(c))
                continue;

            s = G[c-'A'];
            nullable = 1;

            for(k = j+1; p[i][k]; k++) {
                char x = p[i][k];

                if(x == '#')
                    continue;

                if(!isupper(x)) {
                    change |= add(s, x);
                    nullable = 0;
                    break;
                }

                for(int t = 0; F[x-'A'][t]; t++)
                    if(F[x-'A'][t] != '#')
                        change |= add(s, F[x-'A'][t]);

                if(!strchr(F[x-'A'], '#')) {
                    nullable = 0;
                    break;
                }
            }

            if(nullable)
                for(k = 0; G[p[i][0]-'A'][k]; k++)
                    change |= add(s, G[p[i][0]-'A'][k]);
        }
    }
  } while(change);
}

/* Display a set */
void print(char *s)
{
  int i;

  printf("{");
```

```
    for(i = 0; s[i]; i++) {
        if(i) printf(", ");

        if(s[i] == '#')
            printf("ε");
        else
            printf("%c", s[i]);
    }

    printf("}\n");
}

int main()
{
    int i;
    char used[26] = {0};

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions (e.g. S->AB, A->#):\n");

    for(i = 0; i < n; i++) {
        scanf("%19s", p[i]);
        used[p[i][0]-'A'] = 1;
    }

    first();
    follow();

    printf("\nFIRST and FOLLOW sets:\n");

    for(i = 0; i < 26; i++) {
        if(used[i]) {
            printf("FIRST(%c) = ", 'A'+i);
            print(F[i]);

            printf("FOLLOW(%c) = ", 'A'+i);
            print(G[i]);
        }
    }

    return 0;
}
```

```
#include <stdio.h>
#include <string.h>

#define MAX 100

char input[MAX], stack[MAX];
int top = -1, ip = 0, len;

void show(char action[])
{
    int i;

    printf("$");
    for(i = 0; i <= top; i++)
    {
        if(stack[i] == 'i')
            printf("id");
        else
            printf("%c", stack[i]);
    }

    printf("\t\t");

    for(i = ip; i < len; i++)
    {
        if(input[i] == 'i')
            printf("id");
        else
            printf("%c", input[i]);
    }

    printf("$\t\t%s\n", action);
}

int reduce()
{
    /* E -> id */
    if(top >= 0 && stack[top] == 'i')
    {
        stack[top] = 'E';
```

```
        show("REDUCE E->id");
        return 1;
    }

    /* E -> (E) */
    if(top >= 2 &&
       stack[top-2] == '(' &&
       stack[top-1] == 'E' &&
       stack[top] == ')')
    {
        top -= 2;
        stack[top] = 'E';
        show("REDUCE E->(E)");
        return 1;
    }

    /* E -> E*E */
    if(top >= 2 &&
       stack[top-2] == 'E' &&
       stack[top-1] == '*' &&
       stack[top] == 'E')
    {
        top -= 2;
        stack[top] = 'E';
        show("REDUCE E->E*E");
        return 1;
    }

    /* E -> E+E, unless * is next */
    if(top >= 2 &&
       stack[top-2] == 'E' &&
       stack[top-1] == '+' &&
       stack[top] == 'E' &&
       (ip == len || input[ip] != '*'))
    {
        top -= 2;
        stack[top] = 'E';
        show("REDUCE E->E+E");
        return 1;
    }

    return 0;
}
```

```
int main()
{
    char str[MAX];
    int i;

    printf("Grammar: E -> E+E | E*E | (E) | id\n");
    printf("Enter expression: ");
    scanf("%99s", str);

    /* Convert id into i */
    for(i = 0; str[i] != '\0'; i++)
    {
        if(str[i] == 'i' && str[i+1] == 'd')
        {
            input[len++] = 'i';
            i++;
        }
        else
        {
            input[len++] = str[i];
        }
    }

    input[len] = '\0';

    printf("\nSTACK\t\tINPUT\t\tACTION\n");
    printf("-----------------------------------------------\n");
    show("INITIAL");

    while(ip < len)
    {
        /* Reduce as much as possible */
        while(reduce());

        /* Reject adjacent operands */
        if(top >= 0 &&
           (stack[top] == 'E' || stack[top] == ')') &&
           (input[ip] == 'i' || input[ip] == '('))
        {
            printf("String REJECTED.\n");
            return 0;
        }

        /* Shift next symbol */
```

```
        stack[++top] = input[ip++];
        show("SHIFT");
    }

    while(reduce());

    printf("------------------------------------------------\n");

    if(top == 0 && stack[0] == 'E')
        printf("String ACCEPTED.\n");
    else
        printf("String REJECTED.\n");

    return 0;
}
```